STM32F3 Technical Training

For reference only

Refer to the latest documents for details

CRC calculation unit

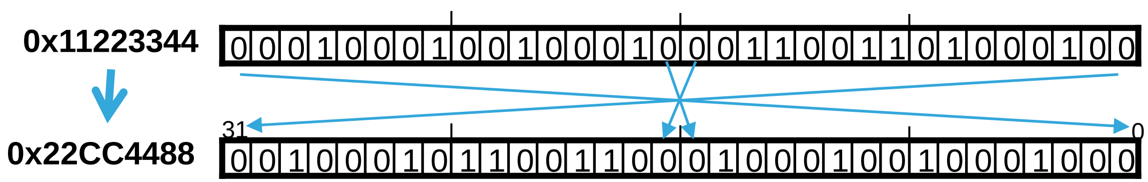
life.augmented

CRC Introduction 1/2

- CRC-based techniques are used to verify data integrity (communications)
- In functional safety standards (such as EN/IEC 60335-1), CRC peripheral offers a means of verifying the embedded Flash memory integrity
- Single input/output 32-bit data register, but handles 8,16, 32-bits input data size
- CRC computation done in 4 AHB clock cycles (HCLK) maximum
- General-purpose 8-bit register (can be used for temporary storage)

CRC Introduction 2/2

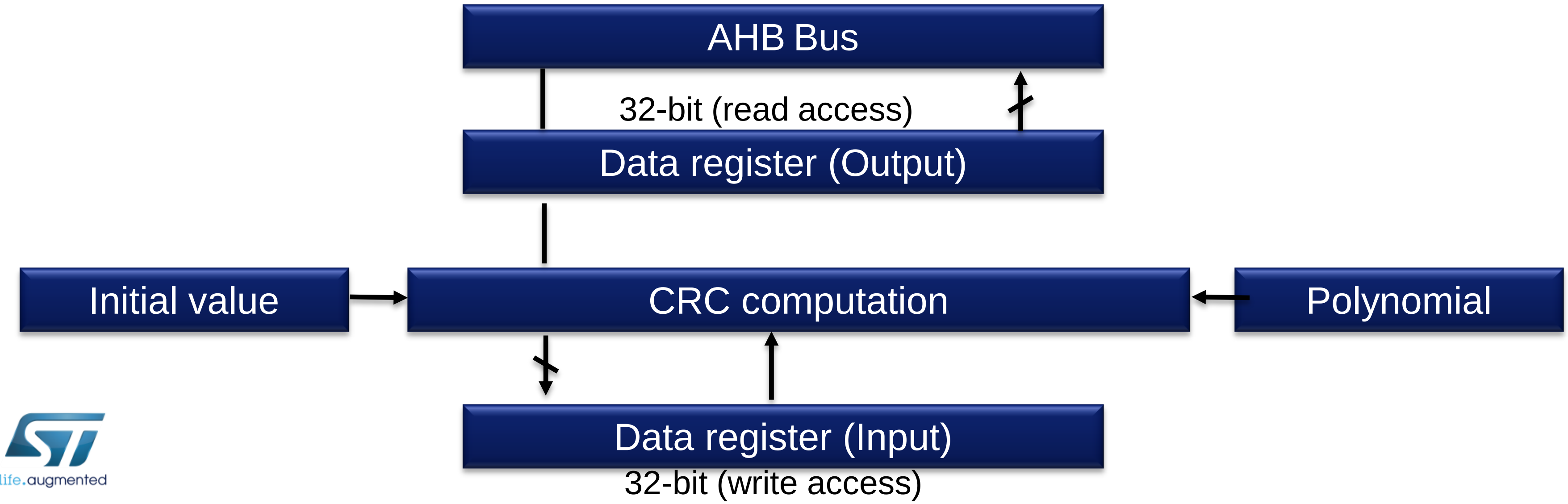
- New features:
  - Programmable parameters:
    - Programmable polynomial:
      - By default uses CRC-32 (Ethernet) polynomial: 0x4C11DB7
      - Alternatively uses a fully programmable polynomial with programmable size (7, 8, 16, 32 bit)
    - Programmable polynomial size (7, 8, 16, 32 bits)
    - Programmable CRC initial value (default = 0xFFFF_FFFF)
  - Reversibility option on I/O data
    - Input data can be reversed by 8, 16, 32 bit
    - Example if input data 0x1A2B3C4D is used for CRC calculation as:
      - 0x58D43CB2 with bit-reversal done by byte
      - 0xD458B23C with bit-reversal done by half-word
      - 0xB23CD458 with bit-reversal done on the full word
    - Output data can be reversed in 32-bit (output register)
      - Example on output data 0x11223344:



life.augmented

CRC Operation

- Operation:
  - Each write operation to the data register creates a combination of the previous CRC value (stored in CRC_DR) and the new one. CRC computation is done on the whole 32-bit data word or byte by byte depending on the format of the data being written.
  - The duration of the CRC computation depends on input data width:
    - 4 AHB clock cycles for 32-bit
    - 2 AHB clock cycles for 16-bit
    - 1 AHB clock cycles for 8-bit
  - Polynomial can be changed after finishing current CRC calculation (or after CRC reset)
  - The input and output data can be bit reversed, to manage the various endianness schemes (REV_IN [1:0], REV_OUT bits).



- What are the new programmable parameters in CRC?
- How many cycles are required to compute a CRC of 15 bytes from RAM ?
- What is the value taken into CRC computation if data input is : **0x11223344** and reversal mode is set to half word ?
  - 0x44332211
  - 0x22114433
  - 0x448822CC

Digital to Analog Converter DAC

life.augmented

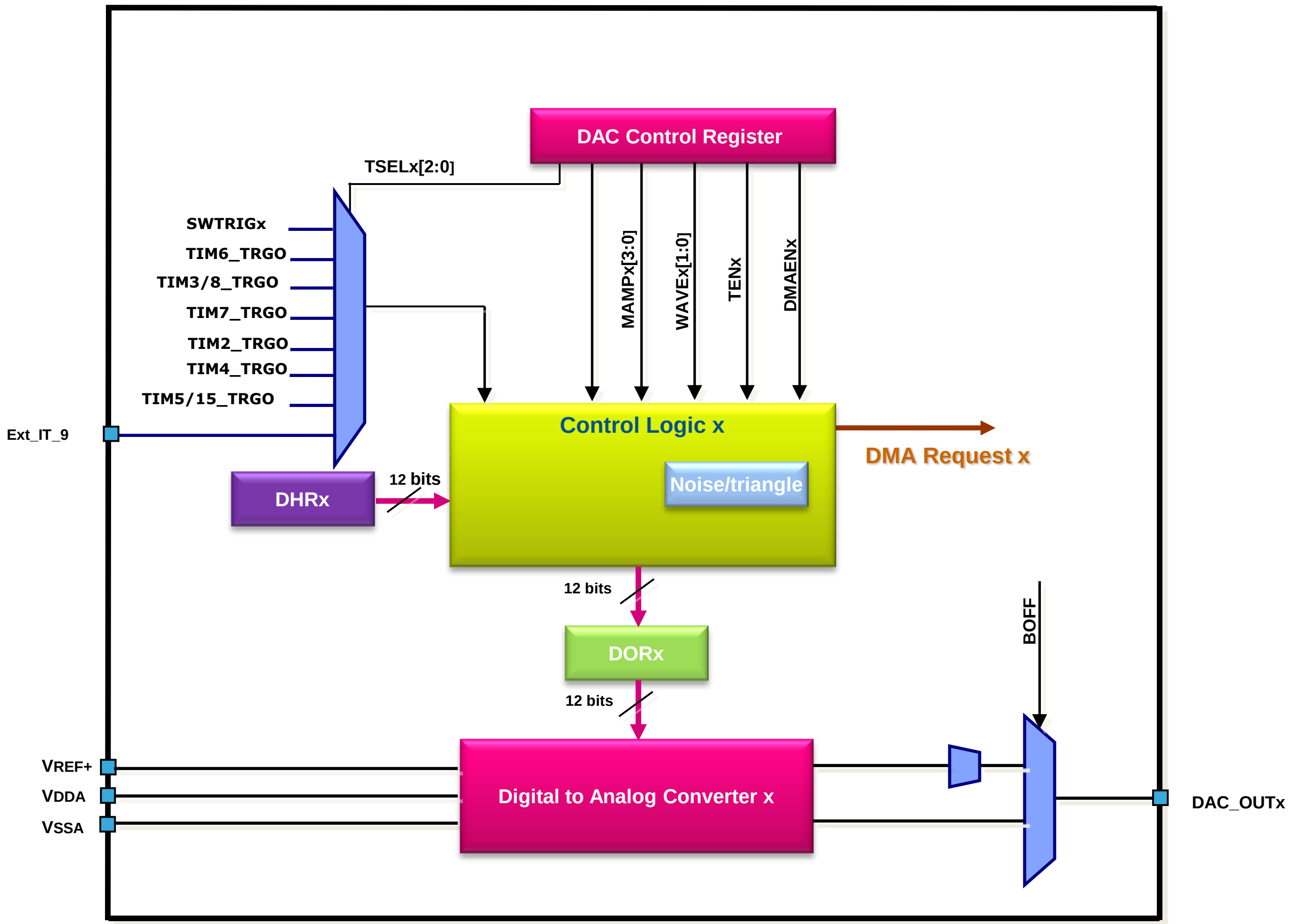
DAC introduction

- Interfaces:
  - Two 12-bit DAC converters inside STM32F37x:
    - DAC1 with 2 DAC output channels
    - DAC2 with 1 output channel
  - One 12-bit DAC converter inside STM32F30x:
    - DAC1 with 2 DAC output channels

- Features and differences:
    - 8-bit or 12-bit mode (left or right data alignment in 12-bit mode)
    - Synchronized update capability
    - Noise-wave or Triangular-wave generation
    - DMA capability for each channel (with DMA underrun error detection)
    - External triggers for conversion (Timers, ext. pin, SW trigger)
    - Programmable output buffer to drive more current
    - Input voltage reference VREF+
    - DAC supply requirement: VDDA = 2.4V to 3.6 V
    - DAC outputs range: 0 ≤ DAC_OUTx ≤ VREF+
    - Dual DAC channel mode supported by DAC1 only:
      - Two channels can be used independently or simultaneously when both channels are grouped together for synchronous update operations (dual mode).

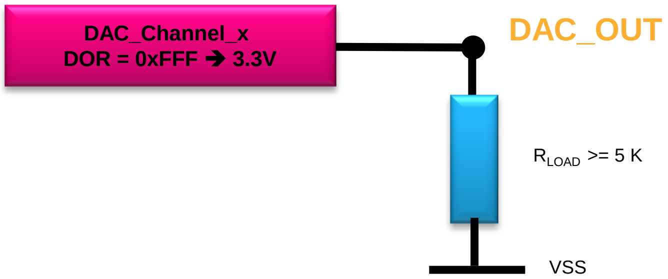
life.augmented

DACx channel block diagram



DAC analog output

- Output voltage:
  - Analog output voltage is given by formula:
    - DAC Output = V_{REF+} * (DOR / 4095)
      - V_{REF+} ……. reference voltage (shared with ADC, input pin or shared with VDDA)
      - DOR ……. Data output register
- Output current:
  - Optional output analog buffer (booster) to improve current capability (BOFF bit)
  - Without output analog buffer (BOFF bit = 1):
    - Rail to rail output: Vout = (V_{REF+} + 1LSB) – (V_{REF+} - 1LSB)
    - Output impedance: 15kΩ
      - Min. load for 1% error: >1.5MΩ
  - With output analog buffer (BOFF bit = 0):
    - Limited output near edges: Vout = (200mV) – (V_{DDA} - 200mV)
    - Min. load for 1LSB error: >5kΩ



life.augmented

DAC data format

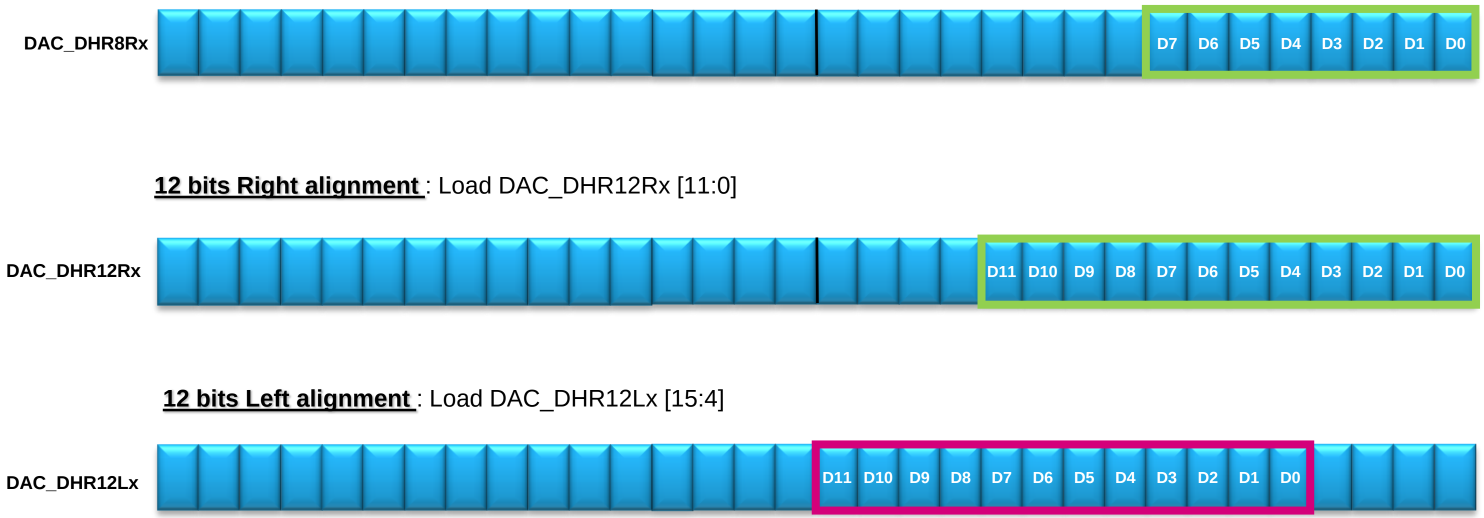
- 8-bit mode:
  - Always right alignment (in register DAC_DHR8Rx)
  - Also in dual channel mode (register DAC_DHR8RD)

- 12-bit mode:
  - Right alignment (in register DAC_DHR12Rx)
  - Left alignment (in register DAC_DHR12Lx)
  - Also in dual channel mode (registers DAC_DHR12RD, DAC_DHR12LD)

8 bits Right alignment: Load DAC_DHR8Rx [7:0]

12 bits Right alignment : Load DAC_DHR12Rx [11:0]

12 bits Left alignment : Load DAC_DHR12Lx [15:4]



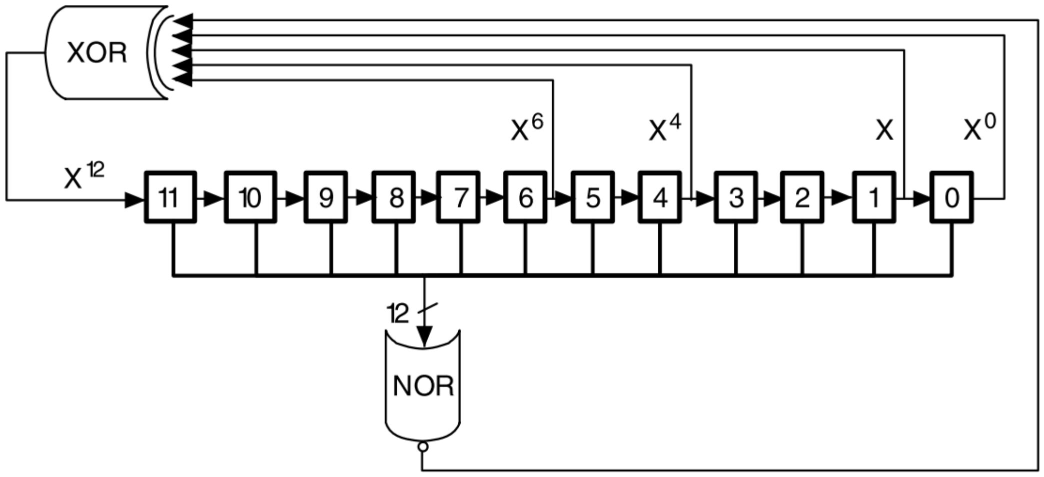
DAC conversion triggers

- Conversion started (load data to the DORx register) by:
  - Automatically (if external trigger disabled TENx) :
    - From DAC_DHRx after one APB1 clock cycle
  - Triggered conversion (if external trigger enabled TENx) :
    - After three APB1 clock cycles after trigger generated by external trigger (except SWTRIG)

- Triggers:
  - Timers:
    - Timer 6 TRGO event
    - Timer 3 TRGO event (or Timer 8 as option for STM32F30x)
    - Timer 7 TRGO event
    - Product dependent :
      - For STM32F37x: Timer 5 (for DAC1) / Timer 18 (for DAC2) TRGO event
      - For STM32F30x: Timer 15
    - Timer 2 TRGO event
    - Timer 4 TRGO event
  - External pin
    - EXTI line9
  - Software
    - SWTRIG bit

DAC noise wave generation

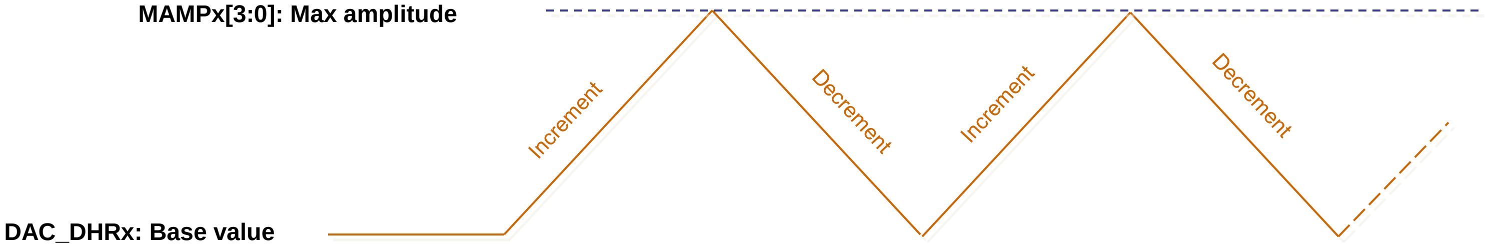
- Noise generation:
  - Based on LFSR (linear feedback shift register):
    - Initial value = 0xAAAA
  - The LFSR 12bits value can be masked partially or totally
  - Anti lock-up mechanism: if LFSR equal to 0 then a 1 is injected on it
  - Calculated noise value (updated through external trigger) is added to the DAC_DHRx content without overflow



life.augmented

DAC triangle wave generation

- Triangle generation:
  - Add a small-amplitude triangular waveform on a DC or slowly varying signal: used as basic waveform generator for example
  - Calculated triangle value (updated through external trigger) is added to the DAC_DHRx content without overflow to reach the configurable max amplitude
  - Up-Down triangle counter:
    - Incremented to reach defined max amplitude value
    - Decremented to return to the initial base value
  - Triangle max. amplitude are configurable: (2^N–1) with N=[1..12]



DAC dual channel mode

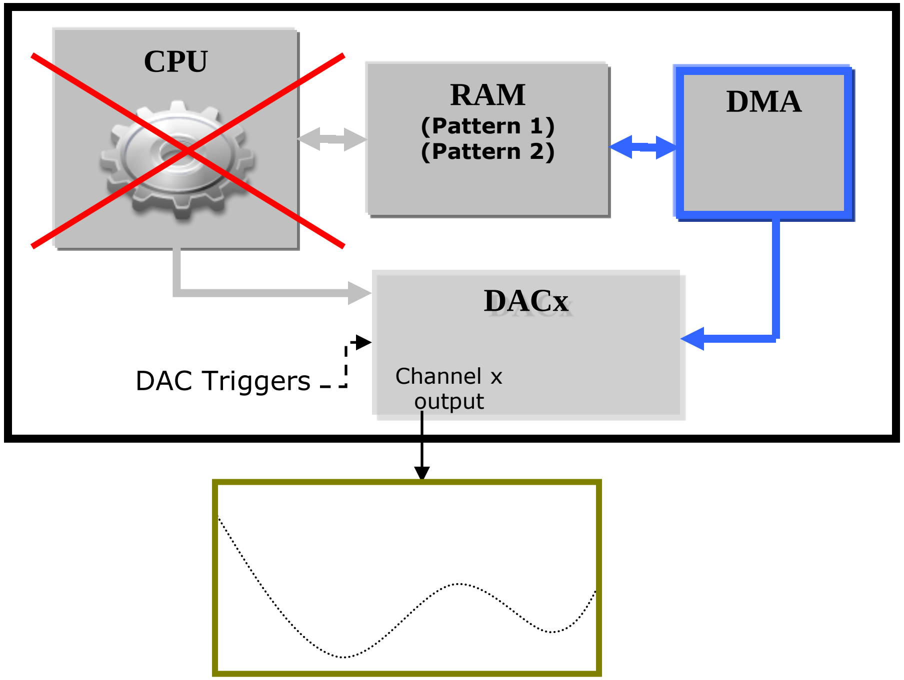
- For DAC1 only (2 channel outputs):
  - Both DAC channels can be used together
    - generate differential or stereo signals in simultaneous conversion mode
  - 11 DAC dual modes:
    - Independent trigger or Simultaneous trigger or Software start
      - without or with wave generation
      - LFSR or Triangle wave generation
    - All modes listed:
      1. Independent trigger without wave generation
      2. Independent trigger with single noise generation
      3. Independent trigger with different noise generation
      4. Independent trigger with single triangle generation
      5. Independent trigger with different triangle generation
      6. Simultaneous trigger without wave generation
      7. Simultaneous trigger with single LFSR generation
      8. Simultaneous trigger with different LFSR generation
      9. Simultaneous trigger with single triangle generation
      10. Simultaneous trigger with different triangle generation
      11. Simultaneous software start

12 bits Left alignment in dual channel mode : Load DAC_DHR12LD – bits [31:20] [15:4]

DAC_DHR12LD	D11	D10	D9	D8	D7	D6	D5	D4	D3	D2	D1	D0					D11	D10	D9	D8	D7	D6	D5	D4	D3	D2	D1	D0				

DAC with DMA

- A DAC DMA request is generated when an external trigger occurs:
  - The value of the DAC_DHR register is then transferred to the DAC_DOR register.
- DMA underrun error detection with interrupt capability



Quiz

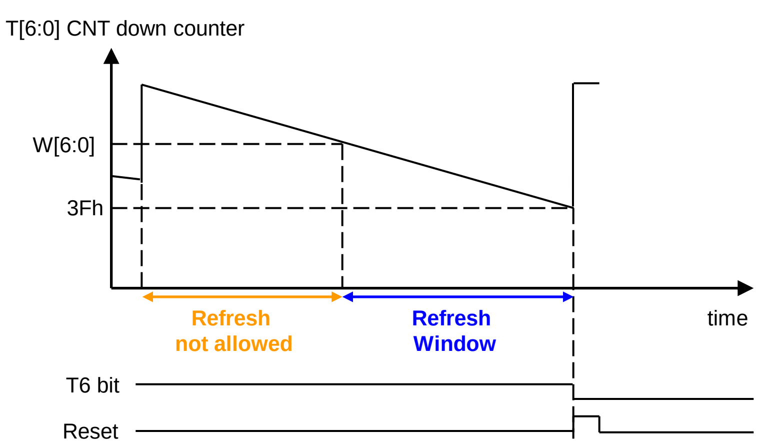
- How many DAC channels are in the STM32F3x microcontroller ?
- What are the possibilities to start a DAC channel conversion?
- What are the different generated waves?
- What is the min. load of DAC channel output?

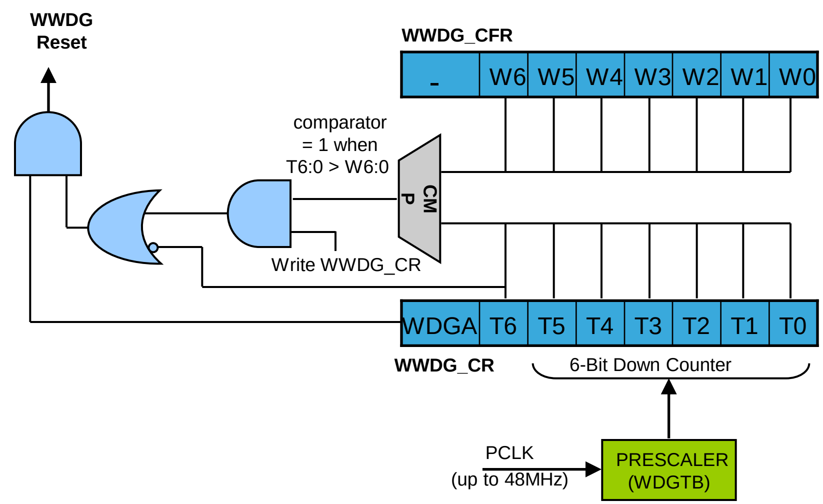
Independent and System Watchdogs

life.augmented

System window watchdog (WWDG) features (1/2)

- Configurable time-window – can be programmed to detect abnormally late or early application behavior
- Conditional reset
  - Reset (if watchdog activated) when the down counter value becomes less than 40h (T6=0)
  - Reset (if watchdog activated) if the down counter is reloaded outside the time-window
- To prevent WWDG reset:
  - write T[6:0] bits (with T6 equal to 1) at regular intervals while the counter value is lower than the time-window value (W[6:0])





WWDG features (2/2)

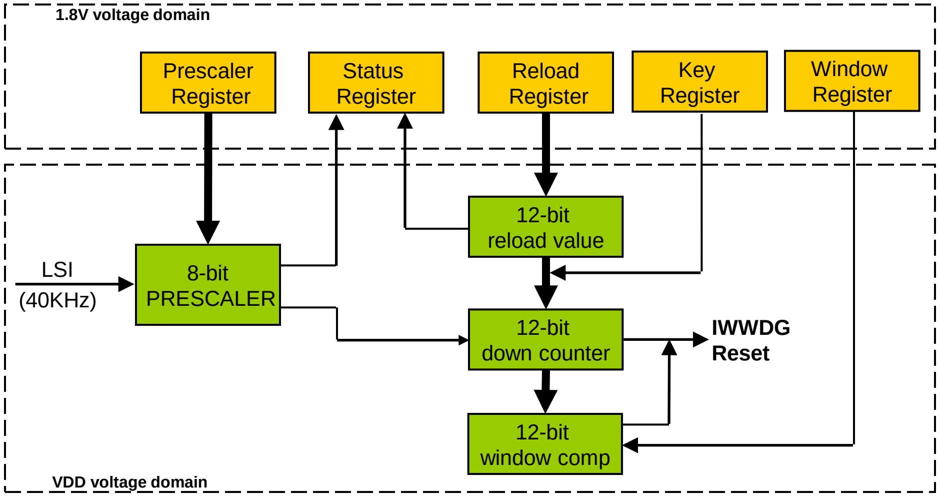
- Early Wakeup Interrupt (EWI):
  - occurs whenever the counter reaches 40h
    - can be used to reload the down counter or recovery or store state before reset (in special cases)

- WWDG reset flag (in RCC_CSR) to inform when a WWDG reset occurs (after device reset)

- Prescaled from the APB1 clock (36MHz):
  - 4 predividers:
    - 4096 * 1
    - 4096 * 2
    - 4096 * 4
    - 4096 * 8
  - Min-max timeout value: 113.8µs / 58.25ms

Best suited to applications which require the watchdog to react within an accurate timing window

Independent window watchdog (IWDG) features (1/2)



Best suited to applications which require the watchdog to run as a totally independent process outside the main application

- Selectable HW/SW start through option byte
- Clocked from an independent RC oscillator (LSI)
  - can operate in Standby and Stop modes
- Once enabled the IWWDG can't be disabled (LSI can't be disabled too)
- Implemented in the VDD voltage domain:
  - Is still functional in STOP and STANDBY mode

life.augmented

Independent window watchdog (IWDG) features (2/2)

- Conditional reset
  - Reset (if IWDG activated) when the downcounter value becomes less than 0x000
  - Reset (if IWDG activated) if the downcounter is reloaded outside the window
- To prevent IWDG reset:
  - write to IWWDG_KR register with 0xAAAA key value at regular intervals before the counter reaches 0, while respecting the defined refresh window
  - Safe Reload Sequence (key) + window
- IWDG reset flag (in RCC_CSR) to inform when a IWDG reset occurs (after device reset)
- Prescaled from the LSI clock (40kHz):
  - 8-bit predivider: 4-256 (and 12-bit watchdog counter):
  - Min-max timeout value: 100µs / 26.2s

Quiz

- Which clock feeds the IWWDG down counter?
- How can be the IWWDG started?
- In which WDG is implemented the window option?
- How to detect that device was reset by watchdogs?

STM32 Releasing your **creativity**

Thank you !

life.augmented